

Compte rendu de TP Java 2ème partie

Nom : Délio Duchesne-Brajon

Groupe : 1

Compte rendu de TP Java 2ème partie	1
Table des matières	1
1. Analyse du problème	2
2. Conception de la solution	2
3. Réalisation	2
Exercice 1 – Message de bienvenue	3
Exercice 2 – Calcul du prix TTC	3
Quiz théorique	4
Exercices sur les identifiants	5
Affichages en console	6
Correction de bugs	10
Cas pratique – Jeu de hasard	13
Cas problème – Affichage de messages encadrés	15
4. Tests	16
5. Retour d'expérience	17
6. Conclusion	17

1

1. Analyse du problème

L'objectif de ce TP est de renforcer les connaissances en programmation Java.
Pour cela, plusieurs activités sont proposées :

- conversion d'algorithmes en Java,
- quiz de compréhension,

1

- exercices de programmation,
- cas pratiques et cas problèmes.

Le but est d'allier théorie et pratique afin d'améliorer la maîtrise du langage Java.

2. Conception de la solution

Les exercices ont été décomposés en plusieurs sous-problèmes :

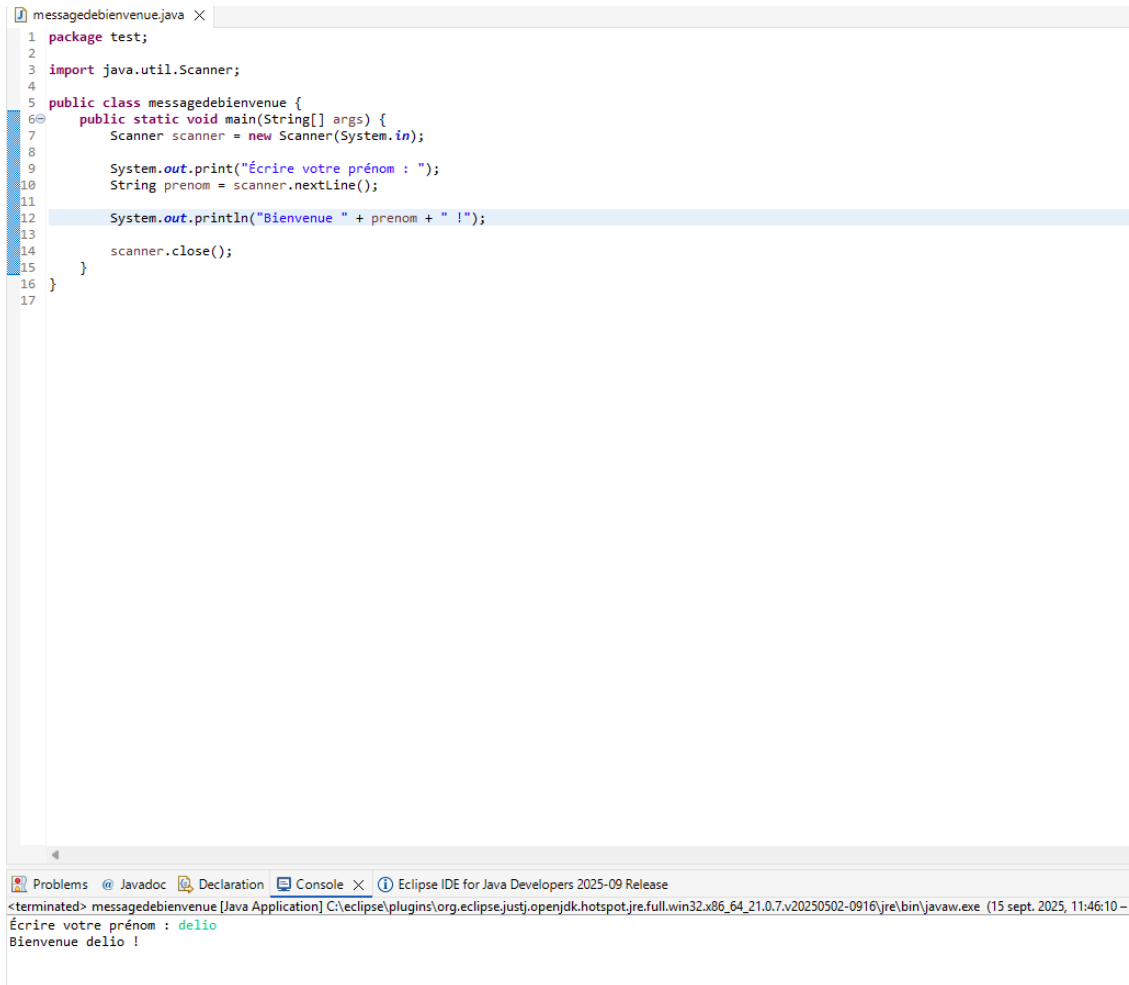
1. Interaction avec l'utilisateur (lecture de saisie et affichage).
2. Calculs simples (prix HT/TTC).
3. Respect des règles de syntaxe Java (identifiants, classes, méthodes).
4. Affichage de formes et de textes.
5. Correction de bugs.
6. Développement de petits programmes interactifs.

3. Réalisation

Chaque exercice a été implémenté dans l'IDE **BlueJ**.
Les codes sources sont fournis en annexe.

Exercice 1 – Message de bienvenue

L'utilisateur saisit son prénom, puis le programme affiche « Bienvenue [prénom] ».



The screenshot shows the Eclipse IDE interface. The top editor window displays a Java file named `messagedebienvenue.java` with the following code:

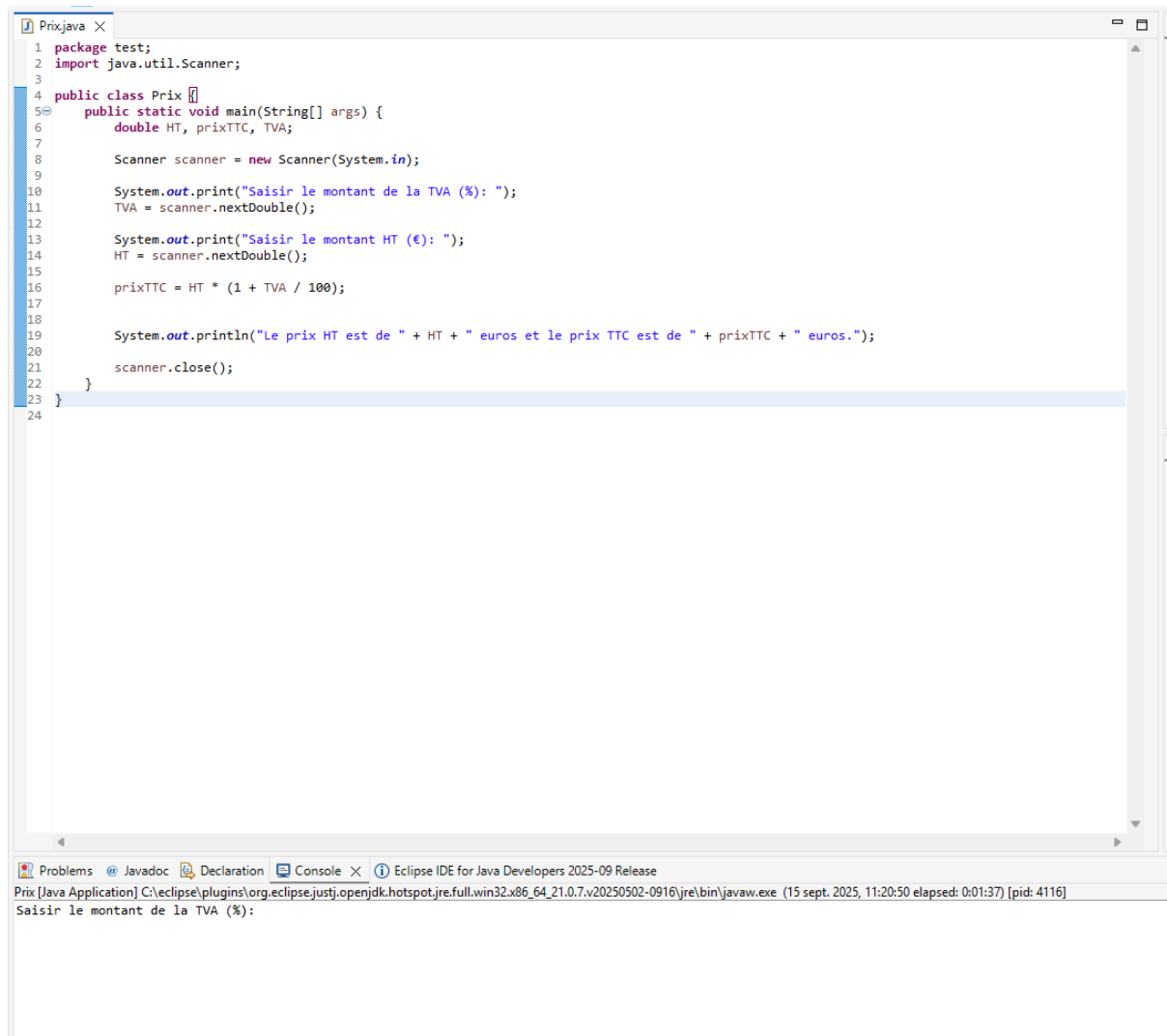
```
1 package test;
2
3 import java.util.Scanner;
4
5 public class messagedebienvenue {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8
9         System.out.print("Écrire votre prénom : ");
10        String prenom = scanner.nextLine();
11
12        System.out.println("Bienvenue " + prenom + " !");
13
14        scanner.close();
15    }
16 }
17
```

The bottom console window shows the output of the program:

```
<terminated> messagedebienvenue [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\javaw.exe (15 sept. 2025, 11:46:10 - Écrire votre prénom : delio
Bienvenue delio !
```

Exercice 2 – Calcul du prix TTC

Le programme demande le montant HT et le taux de TVA, puis calcule le prix TTC.



```
1 package test;
2 import java.util.Scanner;
3
4 public class Prix {
5     public static void main(String[] args) {
6         double HT, prixTTC, TVA;
7
8         Scanner scanner = new Scanner(System.in);
9
10        System.out.print("Saisir le montant de la TVA (%): ");
11        TVA = scanner.nextDouble();
12
13        System.out.print("Saisir le montant HT (€): ");
14        HT = scanner.nextDouble();
15
16        prixTTC = HT * (1 + TVA / 100);
17
18        System.out.println("Le prix HT est de " + HT + " euros et le prix TTC est de " + prixTTC + " euros.");
19
20        scanner.close();
21    }
22 }
23
24
```

Problems Javadoc Declaration Console Eclipse IDE for Java Developers 2025-09 Release

Prix [Java Application] C:\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\javaw.exe (15 sept. 2025, 11:20:50 elapsed: 0:01:37) [pid: 4116]

Saisir le montant de la TVA (%):

Quiz théorique

- Le langage machine le plus basique niveau circuit est
a. Le langage machine
- Les langages qui permettent d'utiliser un vocabulaire qui utilise les termes : read, write ou add sont :
b. Haut niveau
- Les règles du langage de programmation constituent :
a. la syntaxe
- Un _____ traduit les instructions de langage de haut niveau en code machine
c. un compilateur
- Les emplacements de mémoire nommés de l'ordinateur sont appelés :
b. variables
- Les opérations individuelles utilisées dans un programme informatique sont souvent regroupées en unités logiques appelées :

a. procédures

- Une instance de classe est :
c. un objet
- Java a une architecture
a. neutre
- Vous devez compiler les classes écrites en Java dans
a. un bytecode
- Toutes les instructions de programmation Java doivent se terminer par :
c. un point-virgule
-

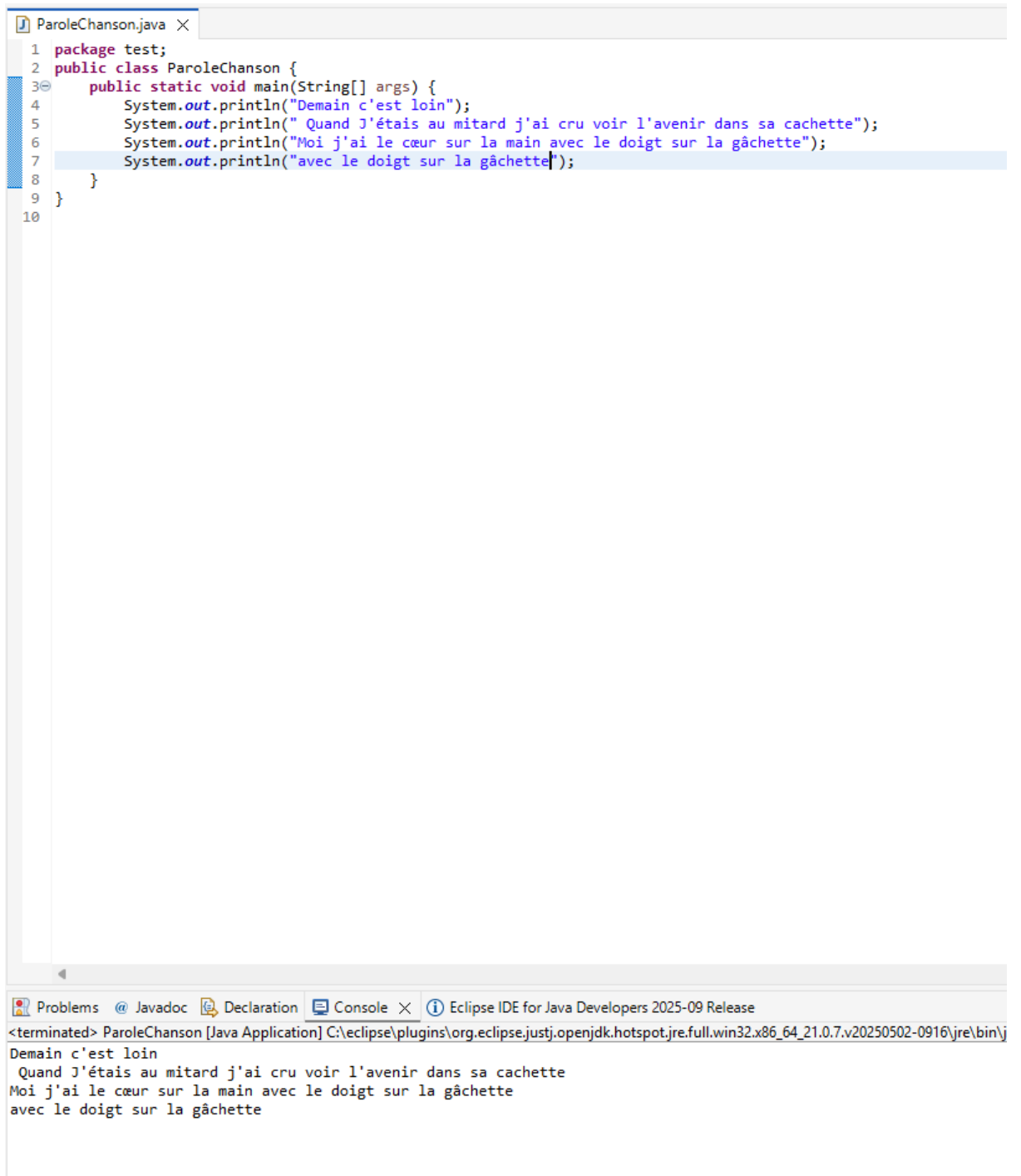
Exercices sur les identifiants

Identifiant	Légal ?	Conventionnel ?	Explication
maClasse	Oui	Non	Valide, mais commence par une minuscule (non conventionnel pour une classe).
void	Non	Non	Mot réservé du langage Java, interdit comme nom de classe.
Golden Retriever	Non	Non	Contient un espace, ce qui est interdit.
invoice#	Non	Non	Contient un caractère spécial (#) non autorisé.
36535CodePostal	Non	Non	Commence par un chiffre, interdit.
Appartement	Oui	Oui	Valide, commence par une majuscule, respecte la convention.
Fruit	Oui	Oui	Valide, clair et conventionnel.
8888	Non	Non	Identifiant purement numérique, non valide.
EcranTotal()	Non	Non	Contient des parenthèses, interdit dans un identifiant.
Acompte_recevable	Oui	Non	Valide mais les underscores sont rarement utilisés pour les noms de classe. Convention : utiliser CamelCase.

Identifiant	Légal ?	Conventionnel ?	Explication
associationRoles()	Oui	Oui	Valide, commence par une minuscule, suit la convention camelCase.
void()	Non	Non	Mot réservé du langage Java, interdit comme nom de méthode.
Golden Retriever()	Non	Non	Contient un espace, ce qui est interdit dans un identifiant.
invoice#()	Non	Non	Contient un caractère spécial (#), interdit.
24500CodePostal()	Non	Non	Commence par un chiffre, ce qui est interdit.
PayrollApp()	Oui	Non	Valide, mais commence par une majuscule (non conventionnel pour une méthode).
getReady()	Oui	Oui	Valide, suit bien la convention camelCase.
911()	Non	Non	Identifiant purement numérique, interdit.
EcranTotal()	Oui	Non	Valide, mais commence par une majuscule (plutôt réservé aux classes).
Acompte_Recevable()	Oui	Non	Valide, mais l'usage des underscores et des majuscules ne suit pas la convention camelCase.

Affichages en console

- Texte (paroles de chanson).



The screenshot shows the Eclipse IDE interface. The top editor window displays the file `ParoleChanson.java` with the following code:

```
1 package test;
2 public class ParoleChanson {
3     public static void main(String[] args) {
4         System.out.println("Demain c'est loin");
5         System.out.println(" Quand J'étais au mitard j'ai cru voir l'avenir dans sa cachette");
6         System.out.println("Moi j'ai le cœur sur la main avec le doigt sur la gâchette");
7         System.out.println("avec le doigt sur la gâchette");
8     }
9 }
10
```

The bottom console window shows the output of the program:

```
<terminated> ParoleChanson [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\j
Demain c'est loin
Quand J'étais au mitard j'ai cru voir l'avenir dans sa cachette
Moi j'ai le cœur sur la main avec le doigt sur la gâchette
avec le doigt sur la gâchette
```

Ici on affiche simplement 4 lignes différentes des 4 lignes du couplet de mister you « guide de la débrouille » C'était la musique que j'écoutait au moment de faire le test. Cela donne ce résultat

- Dessin ASCII (table + chaises).

```

TablesEtChaises.java ×
1 package test;
2
3 import java.util.Scanner;
4
5 public class TablesEtChaises{
6     public static void main(String[] args) {
7         ;
8
9         System.out.println("x           x ");
10        System.out.println("x           x ");
11        System.out.println("x           x ");
12        System.out.println("x      xxxxx  x ");
13        System.out.println("x    x x x  x ");
14        System.out.println("xxxx x  x  xxxx ");
15        System.out.println("x x x x x x x ");
16        System.out.println("x x x x x x x ");
17        System.out.println("x x x x x x x ");
18    }
19 }

```

Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2025-09 Release

<terminated> chanson [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-0916\jre\bin\javaw.exe (1

```

x           x
x           x
x           x
x           x
x      xxxxx  x
x    x x x  x
xxxx x  x  xxxx
x x x x x x x
x x x x x x x
x x x x x x x
x x x x x x x

```

ici on fait le dessin ligne par ligne avec des X placés au bon endroit, ce qui donne ce résultat dans la console :

- Triangle en T.

Triangles.java X

```
1 package test;
2 public class Triangles {
3     public static void main(String[] args) {
4         System.out.println("    T");
5         System.out.println("   TTT");
6         System.out.println("  TTTTT");
7         System.out.println(" TTTTTT");
8         System.out.println("TTTTTTTT");
9         System.out.println("TTTTTTTTT");
10        System.out.println("TTTTTTTTTT");
11    }
12 }
13
```

Problems @ Javadoc Declaration Console X Eclipse IDE for Java Developers 2025-09 Release
<terminated> Triangles [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.

```
  T
 TTT
TTTTT
TTTTTT
TTTTTTTT
TTTTTTTTT
TTTTTTTTTT
TTTTTTTTTTT
```

Ici le plus compliqué car il faut calculer l'espace entre chaque "T"

Correction de bugs

Plusieurs extraits de code erronés ont été corrigés :

- erreurs de syntaxe,
- fautes d'orthographe dans les instructions (`println`, etc.),
- erreurs de majuscules/minuscules.



```
1 package test;
2
3 public class Debug1 {
4
5     public static void main(String[] args) {
6         System.out.println("Salut");
7     }
8 }
9
```

Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2025-09 Release

<terminated> Debug1 [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502\jre\bin\java.exe

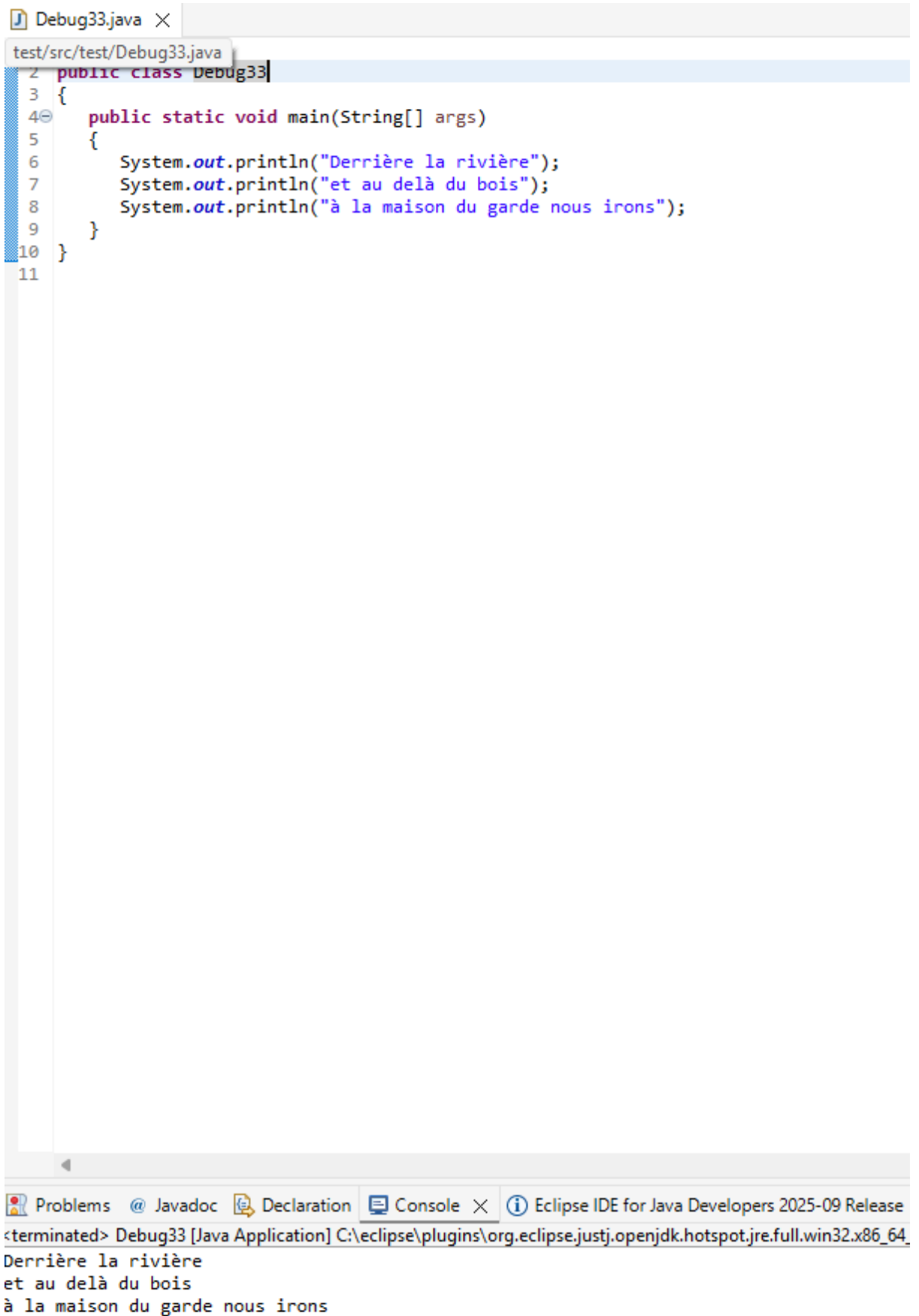
Salut

```
1 package test;
2
3 public class Debug2 {
4
5     public static void main(String[] args) {
6         System.out.println("Programmer en Java est fun.");
7         System.out.println("Faire un programme");
8         System.out.println("peut être un challenge,");
9         System.out.println("mais quand la syntaxe est correcte,");
10        System.out.println("c'est satisfaisant");
11    }
12 }
13
```

Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2025-09 Release

<terminated> Debug2 [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.7.v20250502-09

Programmer en Java est fun.
Faire un programme
peut être un challenge,
mais quand la syntaxe est correcte,
c'est satisfaisant



```
1  Debug33.java ×
2  test/src/test/Debug33.java
3  public class Debug33
4  {
5      public static void main(String[] args)
6      {
7          System.out.println("Derrière la rivière");
8          System.out.println("et au delà du bois");
9          System.out.println("à la maison du garde nous irons");
10     }
11 }
```

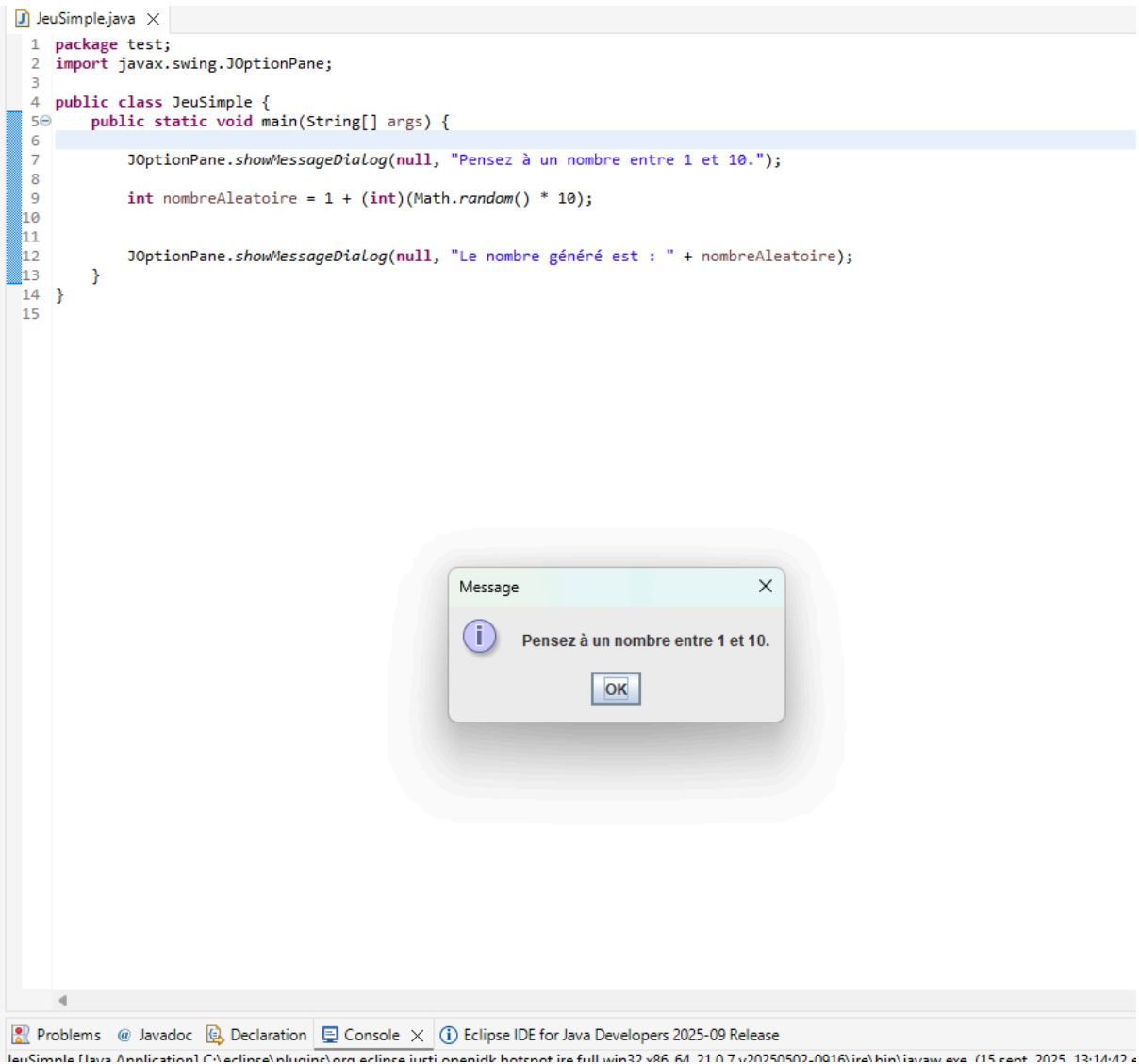
Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2025-09 Release

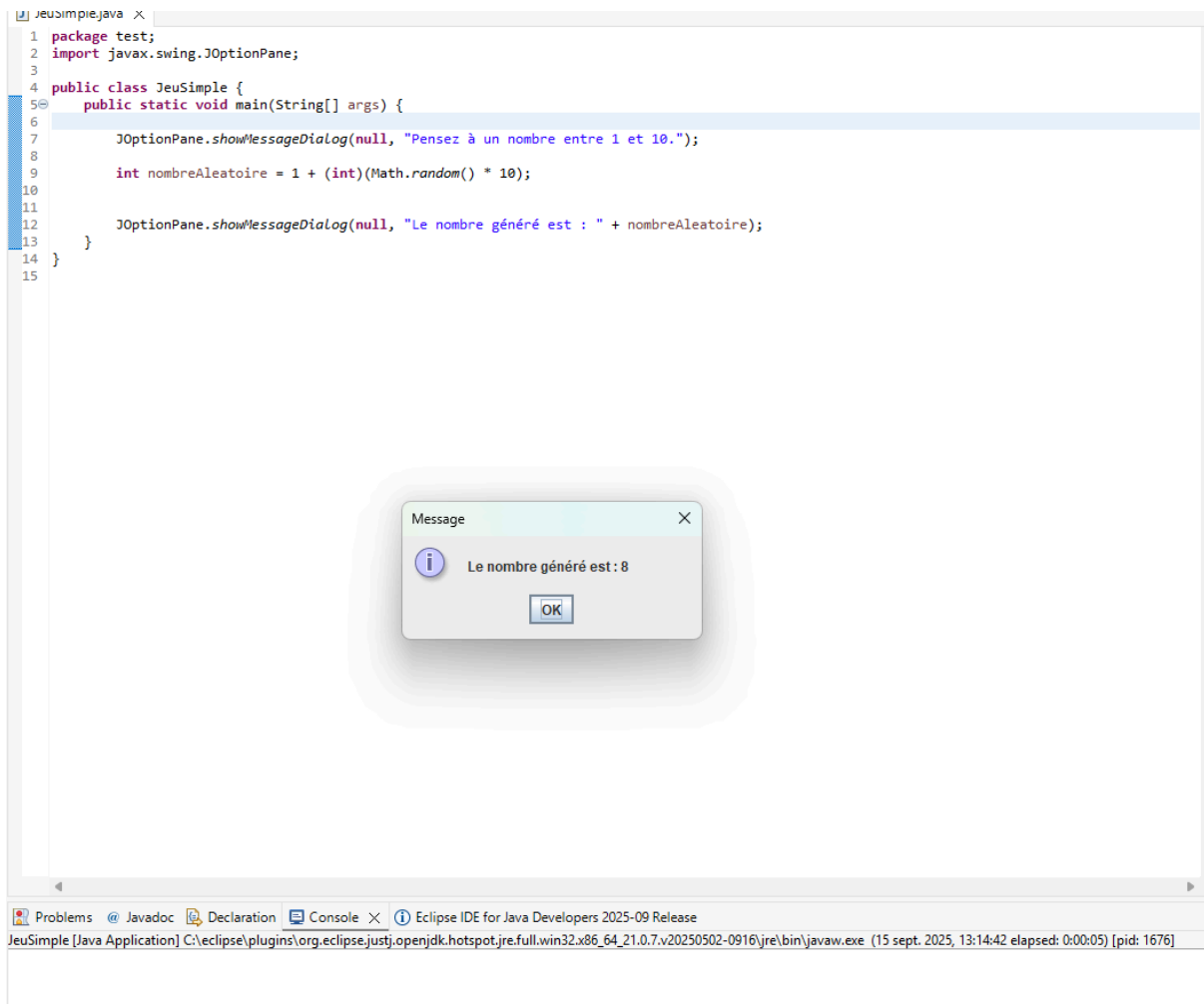
<terminated> Debug33 [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64

Derrière la rivière
et au delà du bois
à la maison du garde nous irons

Cas pratique – Jeu de hasard

Jeu où l'utilisateur pense à un nombre entre 1 et 10, puis un nombre aléatoire est généré.





Cas problème – Affichage de messages encadrés

Trois versions du même message :

- simple,
- entouré d'astérisques,
- entouré de lettres « S ».

```
1 package yummy;
2
3 public class Yummy {
4     public static void main(String[] args) {
5         System.out.println("Yummy prépare les meilleurs plats pour vos fêtes");
6     }
7 }
```


5. Retour d'expérience

Ce TP m'a permis de progresser dans la compréhension de la syntaxe Java et la correction d'erreurs fréquentes.

Les principales difficultés rencontrées :

- la rigueur de la syntaxe (majuscules, guillemets, points-virgules),
- la correction des erreurs de compilation parfois nombreuses.

En revanche, les exercices variés m'ont permis de mieux assimiler les conventions et la logique Java.

6. Conclusion

Ce TP a permis de consolider les acquis en Java en passant de la théorie à la pratique.

Grâce aux exercices progressifs (quiz, programmation, cas pratiques), mes compétences en Java se sont élargies, notamment en termes de rigueur et de méthode.